

so that you don't need to add backslash multiple times. So the following two regular expressions: `pat = re.compile('\\\\t')` and `pat = re.compile(r'\\t')` will match the same pattern `\\t`.

The symbol `*` results in the matching of zero or more repetitions of the preceding regular expression. The regular expression `ab*` will match all the strings starting with an `a` and ending with zero or more `b`'s. The set of all strings matched by the regular expression is `{a, ab, abb, abbb, ...}`. The symbol `+` results in the matching of one or more repetitions of the preceding regular expression. The regular expression `ab+` will match all the strings starting with an `a` and ending with one or more `b`'s. The set of all strings matched by the regular expression is `{ab, abb, abbb, ...}`. The difference between the two is that `ab*` will match the single character string `a`, whereas `ab+` will not match this string. The symbol `?` results in the matching of zero or one repetition of the preceding regular expression. The regular expression `ab?` will match the strings `a` and `ab`.

The two symbols `[]` and `{}|` are used to denote a character class. For example, `[abc]` will match all strings having the letters `a`, `b` or `c`. A hyphen can be used to denote a set of characters. The regular expression `[a-z]` matches all strings having lower case letters. Inside the square brackets used for specifying the character class, all the special characters will lose their special meaning. `[ab*]` matches strings containing the characters `a`, `b` or `*`.

The caret symbol `^` has two purposes. First, it checks for a match at the beginning of a string. `^a` matches all the strings starting with an `a`. Second, the caret symbol inside square brackets means negation. `^[^a]` matches all the lines that start with a character other than `a`. So, a line like `aaabbb` will not be matched whereas a line like `bbbbaaa` will be matched. The symbol `$` matches at the end of a string. `a$` will result in the matching of all the strings ending with an `a`.

As explained earlier, the special symbol dot (`.`) results in the match of any character except the new line character `\\n`, and the DOTALL flag of `compile()` will result in a match of even a new line character. `a.c` will match strings like `aac`, `abc`, `acc`, `a9c`, etc. The symbol `|` is the *or* operator of a regular expression. `black|white` will match the strings with the sub-strings, black or white. So, strings like blackboard, whitewash, black & white, etc., will be matched by the regular expression.

The special symbols, opening and closing curly brackets, are used for searching repeating patterns. This is the one notation that has confused many people who use regular expressions. I would like to analyse why this occurs. Every textbook and article on regular expressions declares that the regular expression `a{m}` matches all the patterns with `m` number of `a`'s, and rightly so. Now consider the contents of the text file `file2.txt`.

```
a
aa
aaa
aaaa
aaaaa
```

Let us have the following pattern: `pat = re.compile('a{3}')` executed on the Python terminal and then call our script `run.py` to do the rest. You might expect to see just one line selected, the line `aaa`. But the output in Figure 3 shows you that the lines, `aaa`, `aaaa` and `aaaaa` are also matched by this pattern because the string `aaa` is printed thrice. The textbook definition kind of suggests to you that only `aaa` should be matched but you are getting much more than that selected. Most of the textbooks that deal with regular expressions fail to explain this anomaly and that is the one point I would like to clarify once and for all, in this article, if nothing else. Theoretical Computer Science 101 says that finite automata do not have the ability to count. Regular expressions and finite automata are different ways of describing the same thing. I can't explain this any further but you have to believe me on this. Now what is the reason for three lines getting selected instead of the single line `aaa`? If you look at the two additional lines selected, `aaaa` and `aaaaa`, both contain the sub-string, `aaa`. That again tells us regular expressions are not counting; instead, they match for patterns and nothing more.

```
>>> import re
>>> pat = re.compile('a{3}')
>>> execfile('run.py')
Enter File Name to Process: file2.txt
aaa
aaa
aaa
```

Figure 3: Counting not possible with regular expressions

The other possibilities with this notation are `a{m}` which searches for patterns with `m` or less number of `a`'s; `a{m,}` which searches for patterns with `m` or more `a`'s; and `a{m,n}` which searches for patterns with `m` to `n` number of `a`'s, where `m` can be any integer constant. But do remember that just like `a{m}`, the regular expressions `a{m}` and `a{m,n}` will also lead to counter intuitive results due to the same reasons mentioned earlier.

The last two special symbols to be explained are the opening and closing parenthesis. These are used to indicate the start and end of a group. For example, `(abc)+` will match strings like `abc`, `abcbac`, `abcabcabc`, etc. The contents of a group can be retrieved after a match, and can be used to match with the later parts of a string with the `\\number` special sequence. The `groups()` method of the match object left unexplained earlier can also be discussed now. Let us assume we are searching for a pattern where three two-digit numbers are separated by a colon, like 11:22:33, 44:55:66, etc. Then one possible regular expression is `(\\d\\d):(\\d\\d):(\\d\\d)`. The text file `file3.txt` contains the following text.

```
11:22:33
aa:bb:cc
dd:cc:ee
44:55:66
```